Total Points

105 / 105 pts

Question 1

Q1

10 / 10 pts

✔ **– 0 pts** Correct understanding Purpose of RAM

– 3 pts Mostly Correct Purpose of RAM with minor inaccuracies or slight vagueness

– 10 pts Incorrect / Blank / Does not state purpose

Question 2

Q2

10 / 10 pts

✔ **– 0 pts** Correct

– 10 pts Incorrect/Vague

Question 3

Q3

10 / 10 pts

✔ **– 0 pts** Correct

– 4 pts Does not describe the nature of the device

– 10 pts Incorrect/Vague

Question 4

Q4

15 / 15 pts

✔ **+ 15 pts** Correct

+ 5 pts Incorrect attempt with some missteps

+ 3 pts Mostly incorrect attempt

– 3.5 pts Moderate Error

– 2 pts Minor calculation error

– 1.5 pts Missing some units / errors with units used

– 1 pt Minor math error

+ 0 pts Incorrect

Question 5

Q5

15 / 15 pts

✔ **– 0 pts** Correct

– 5 pts Almost Correct

– 10 pts Very Vague/ Part Correct

– 15 pts Incorrect

Question 6

Q6

15 / 15 pts

✔ **– 0 pts** Correct

– 1 pt small mistake

– 15 pts incorrect

– 15 pts use of pseudoinstruction

– 1 pt bad syntax

– 4 pts redundant load/store

– 2 pts assuming register is zero

– 2 pts wrong order of operations (see parentheses)

– 4 pts malformed load/store instruction

– 2 pts wrong instruction (addi vs add, for example)

– 15 pts generally incorrect

– 2 pts subtract immediate (pseudoinstruction, but understandable mistake)

– 3 pts incorrect operation

– 3 pts malformed add/addi instruction

– 4 pts malformed instruction that's not add/addi, or load/store

Question 7

Q7

15 / 15 pts

✔ **– 0 pts** Correct

– 4 pts Didn't load value from address stored in $s1

– 2 pts Incorrect loading/storing instruction

– 1 pt Lwu/swu

– 2 pts Loaded into the same register as the one storing address

– 4 pts Didn't store back value

– 3 pts Mistook address as value

– 3 pts Added before conditional

– 2 pts Assuming register == 0 before initialization

– 2 pts Used an instruction wrong/Used wrong instruction

– 2 pts Wrong register/value

– 3 pts Didn't increment i

– 3 pts Missing jump

– 3 pts See comment

– 4 pts See comment

– 10 pts Mostly Wrong

– 15 pts Incorrect

– 15 pts Psuedoinstruction

Question 8

Q8

15 / 15 pts

✔ **– 0 pts** Correct

– 7 pts one is wrong (wrong value or wrong number of bits)

– 15 pts both wrong (wrong value or wrong number of bits)

– 0 pts we use big-endian approach in our course

1. What is the purpose of RAM memory? Make sure you mention specific purpose of RAM that distinguishes it from other memory devices.

RAM memory is a memory device that stores data & instructions of the currently running program. It is faster than storage devices like a hard disk. It is also volatile while storage devices are not.

2. Programs on computers with 32-bit addresses are not able to use more than 4GB of RAM. Why is that? (don't account for word-addressable memory or segmentation technique if you happen to know about it)

32 bit addresses can only represent up to 2^{32} unique address. This means if there was more than 4 GB of RAM, there would be no way to label these additional spots in memory since you can't count that high with 32 bits.

3. What is *transistor*?

It is a device made of semiconductors like silicone that can either let electricity flow through it or it can stop electricity from flowing through it. It is useful because we can interpret a high signal from transistor as a 1 in binary and low signals (not 0 signal b/c of transistor leakage) as a 0 in binary.

4. What is the average CPI of a program in the scenario described below?

   CPU frequency: 2.5 GHz $\Rightarrow 2.5 \times 10^9$
   Dynamic instruction count: $2*10^9$ instruction
   Execution time: 10 second

   Support your answer with mathematical computations. Use measurement units. Without the correct mathematical explanation, the answer is worth 0 points.

total_cycles_occurred = CPU_Frequency * Execution_Time

$= 2.5 \times 10^9 \frac{\text{cycles}}{\text{second}} \cdot 10 \text{ second}$

$= 2.5 \times 10^{10}$ cycles

$$\text{average\_CPI} = \frac{\text{total\_cycles\_occurred}}{\text{dynamic\_instruction\_count}}$$

$$= \frac{2.5 \times 10^{10} \text{ cycles}}{2 \times 10^{9} \text{ instructions}}$$

$$= \frac{25 \text{ cycles}}{2 \text{ instruction}} = 12.5 \text{ cycles per instruction}$$

The average CPI is 12.5 cycles per instruction

5. int num = 5;
   int *ptr = #

   What is the content of *ptr*? You can state a specific content value or just describe the content in a plain English.

Ptr stores the memory address of num. At that memory address, the number 5 is stored.

6. Write a piece of MIPS code that computes the following expression

   foo = 12 + (3 – foo) – 1

   The value of *foo* is already in $s3 register. After all the computations are finished, a new value of *foo* should also be in $s3.
   Don't simplify the expression. Follow the mathematical order of operations (precedence).

```
addi, $t0, $zero, 3
sub  $t0, $t0, $s3  // (3-foo) is in $t0
addi $t1, $zero, $ 12
add  $t1, $t1, $ t0  // 12 + (3 - foo) is in $t1
addi $s3, $t1, -1  // 12+(3-foo)-1 is in $s3
```

7. Convert to MIPS the following expression:

```
for( int i=10; i<100; i++)
    sum += i;
```

↳ init i to 10
while i < 100
inc. i @ end of loop

sum is an unsigned int and its address is stored in $s1.

```
addi $t0, $zero, 10   // init i ($t0) as 10
lw   $t1, 0($s1)      // store sum in $t1
LOOP:
SLTI $t2, $t0, 100    // check if i < 100
BEQ  $t2, $zero, EXIT // if it is not ($t2 stores 0), then
                      // exit loop
add  $t1, $t1, $t0    // add i to sum
addi $t0, $t0, 1      // increment i
j LOOP                // jump back to start of loop

EXIT:
SW   $t1, 0($s1)      // update sum
```

8. Below is a picture of a portion of RAM memory

Content	Address
0000 0001	0x0DAA0000
0000 0010	0x0DAA0001
0000 0100	0x0DAA0002
0000 1000	0x0DAA0003
0001 0000	0x0DAA0004
0010 0000	0x0DAA0005
0100 0000	0x0DAA0006
1000 0000	0x0DAA0007
0000 0011	0x0DAA0008
0000 0110	0x0DAA0009
0000 1100	0x0DAA000A
0001 1000	0x0DAA000B
0011 0000	0x0DAA000C
0110 0000	0x0DAA000D
1100 0000	0x0DAA000E
0000 0111	0x0DAA000F
0000 1110	0x0DAA0010
0001 1100	0x0DAA0011
0011 1000	0x0DAA0012
0111 0000	0x0DAA0013
1110 0000	0x0DAA0014
0000 1111	0x0DAA0015
0001 1110	0x0DAA0016
0011 1100	0x0DAA0017
0111 1000	0x0DAA0018
1111 0000	0x0DAA0019
0000 0101	0x0DAA001A

word starting from -4($s1)

half-word starting from 6($s1)

$s1 contains the value 0x0DAA0008. Show the full binary pattern in $s2 after the following instructions:

A) LW $s2, -4($s1)

0001 0000 0010 0000 0100 0000 1000 0000

B) LHU $s2, 6($s1)

0000 0000 0000 0000 1100 0000 0000 0111